

Punto 5.9 — Patrón Repositorio y Unidad de Trabajo en aplicaciones empresariales

Práctica

Corrección técnica canónica: EF Core ya ofrece en `DbContext` y `DbSet` capacidades relacionadas con Unit of Work y Repository. La abstracción adicional usada por `AceriaData` es una decisión arquitectónica del curso, no una regla universal. No se afirma que Repository sea siempre mejor ni que `DbContext` deba estar siempre oculto.

Ejercicio: Consolidar el patrón Repositorio y la unidad de trabajo en el proyecto `AceriaData`. Refactorizar el repositorio para que sea una implementación empresarial completa. Integrar `IDbContextFactory` para servicios de larga duración. Comparar el rendimiento y el consumo de memoria con y sin repositorio. Documentar los anti-patrones con ejemplos de código. Testing del repositorio con mocks.

Contexto del proyecto: En el punto 5.8 se estudiaron las migraciones en equipos, incluyendo los conflictos, las estrategias de resolución y las buenas prácticas. En este punto se profundiza en el patrón Repositorio y la unidad de trabajo en aplicaciones empresariales, que son la base del acceso a datos en la arquitectura limpia. Esta técnica se usará en el punto 5.10 para el logging y el diagnóstico.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

`cd AceriaData` → entra en la carpeta raíz del proyecto. `cd src/AceriaData.Console` → entra en la carpeta del proyecto de consola.

Paso 2: Crear la interfaz genérica IRepository

Crear el archivo `src/AceriaData.Application/Interfaces/IRepository.cs`:

```
namespace AceriaData.Application.Interfaces;

public interface IRepository<T> where T : class
{
    T? ObtenerPorId(int id);
    List<T> ObtenerTodas();
    void Agregar(T entidad);
    void Eliminar(T entidad);
}
```

Línea 1: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres.

Línea 3: `public interface IRepository where T : class` → declara la interfaz genérica.

Línea 5: `T? ObtenerPorId(int id);` → declara el método que obtiene por Id.

Línea 6: `List ObtenerTodas();` → declara el método que obtiene todas.

Línea 7: `void Agregar(T entidad);` → declara el método que agrega.

Línea 8: `void Eliminar(T entidad);` → declara el método que elimina.

Error común: si se olvida la restricción `where T : class`, EF Core no puede usar `context.Set()` porque requiere que `T` sea una clase.

Paso 3: Crear la implementación genérica Repositorio

Crear el archivo src/AceriaData.Infrastructure/Repositories/Repositorio.cs:

```
using AceriaData.Application.Interfaces;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public class Repositorio<T> : IRepository<T> where T : class
{
    protected readonly AceriaDbContext _context;
    protected readonly DbSet<T> _dbSet;

    public Repositorio(AceriaDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public virtual T? ObtenerPorId(int id)
    {
        return _dbSet.Find(id);
    }

    public virtual List<T> ObtenerTodas()
    {
        return _dbSet.ToList();
    }

    public virtual void Agregar(T entidad)
    {
        _dbSet.Add(entidad);
    }

    public virtual void Eliminar(T entidad)
    {
        _dbSet.Remove(entidad);
    }
}
```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 2: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 3: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 5: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 7: public class Repositorio : IRepository where T : class → declara la implementación genérica.

Línea 9: protected readonly AceriaDbContext _context; → campo del DbContext.

Línea 10: protected readonly DbSet _dbSet; → campo del DbSet.

Línea 12: public Repositorio(AceriaDbContext context) → constructor.

Línea 14: _context = context; → asigna el DbContext.

Línea 15: _dbSet = context.Set(); → obtiene el DbSet.

Línea 18: public virtual T? ObtenerPorId(int id) → declara el método.

Línea 20: return _dbSet.Find(id); → busca por clave primaria.

Línea 23: public virtual List ObtenerTodas() → declara el método.

Línea 25: return _dbSet.ToList(); → materializa la consulta.

Línea 28: public virtual void Agregar(T entidad) → declara el método.

Línea 30: `_dbSet.Add(entidad);` → registra la entidad.

Línea 33: `public virtual void Eliminar(T entidad)` → declara el método.

Línea 35: `_dbSet.Remove(entidad);` → marca la entidad para eliminar.

Error común: si se usa `_context.Set()` en cada método en lugar de almacenarlo en el campo `_dbSet`, se realizan llamadas repetidas al contexto. Es más eficiente almacenarlo en el constructor.

| Paso 4: Refactorizar `IOrdenRepositorio` para heredar de `IRepositorio`

Modificar `src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio : IRepositorio<OrdenFabricacion>
{
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientes();
    List<OrdenFabricacion> ObtenerPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenResumenDto> ObtenerResumenes();
    int ContarOrdenes();
    bool ExisteAlgunaOrden();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude();
    List<OrdenFabricacion> ObtenerConAleacionesThenInclude();
    List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery();
    OrdenFabricacion? ObtenerPorIdParaActualizar(int id);
    bool ActualizarClienteConClienteGana(OrdenFabricacion orden, string nuevoCliente);
    bool InsertarDosOrdenesConTransaccion(string numero1, string numero2);
    void AplicarMigracionHasta(string nombreMigracion);
}
```

Línea 1: `using AceriaData.Application.Dtos;` → importa los DTOs.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 4: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres.

Línea 6: `public interface IOrdenRepositorio : IRepositorio` → declara la interfaz específica que hereda de la genérica.

Línea 8: `OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);` → declara el método.

Línea 9: `List ObtenerPendientes();` → declara el método.

Línea 10: `List ObtenerPorCliente(string cliente);` → declara el método.

Línea 11: `List ObtenerPorEstadoOrdenadasPorFecha(string estado);` → declara el método.

Línea 12: `List ObtenerResumenes();` → declara el método.

Línea 13: `int ContarOrdenes();` → declara el método.

Línea 14: `bool ExisteAlgunaOrden();` → declara el método.

Línea 15: `List ObtenerOrdenesConPlanchasInclude();` → declara el método.

Línea 16: `List ObtenerConAleacionesThenInclude();` → declara el método.

Línea 17: `List ObtenerConPlanchasYDetalleSplitQuery();` → declara el método.

Línea 18: `OrdenFabricacion? ObtenerPorIdParaActualizar(int id);` → declara el método.

Línea 19: `bool ActualizarClienteConClienteGana(OrdenFabricacion orden, string nuevoCliente);` → declara el método.

Línea 20: `bool InsertarDosOrdenesConTransaccion(string numero1, string numero2);` → declara el método.

Línea 21: `void AplicarMigracionHasta(string nombreMigracion);` → declara el método.

Error común: si se añaden demasiados métodos específicos al repositorio, se convierte en una envoltura de todas las consultas posibles. Se deben añadir solo los métodos que la capa de negocio necesita.

| Paso 5: Refactorizar OrdenRepositorio para heredar de Repositorio

Modificar `src/AceriaData.Infrastructure.Repositories/OrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;

namespace AceriaData.Infrastructure.Repositories;

public class OrdenRepositorio : Repositorio<OrdenFabricacion>, IOrdenRepositorio
{
    private readonly ILogger<OrdenRepositorio> _logger;

    public OrdenRepositorio(AceriaDbContext context, ILogger<OrdenRepositorio> logger) : base(context)
    {
        _logger = logger;
    }

    public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden)
    {
        return _context.OrdenesFabricacion
            .FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    }

    public List<OrdenFabricacion> ObtenerPendientes()
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Estado == "Pendiente")
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorCliente(string cliente)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Estado == estado)
            .OrderByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenResumenDto> ObtenerResumenes()
    {
        return _context.OrdenesFabricacion
            .AsNoTracking()
            .Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
            .ToList();
    }
}
```

```

    }

    public int ContarOrdenes()
    {
        return _context.OrdenesFabricacion.Count();
    }

    public bool ExisteAlgunaOrden()
    {
        return _context.OrdenesFabricacion.Any();
    }

    public List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude()
    {
        return _context.OrdenesFabricacion
            .AsNoTracking()
            .Include(o => o.Planchas)
            .OrderBy(o => o.NumeroOrden)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerConAleacionesThenInclude()
    {
        return _context.OrdenesFabricacion
            .AsNoTrackingWithIdentityResolution()
            .Include(o => o.OrdenesAleaciones)
            .ThenInclude(oa => oa.Aleacion)
            .OrderBy(o => o.NumeroOrden)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery()
    {
        return _context.OrdenesFabricacion
            .AsNoTrackingWithIdentityResolution()
            .Include(o => o.Planchas)
            .Include(o => o.Detalle)
            .AsSplitQuery()
            .OrderBy(o => o.NumeroOrden)
            .ToList();
    }

    public OrdenFabricacion? ObtenerPorIdParaActualizar(int id)
    {
        return _context.OrdenesFabricacion
            .FirstOrDefault(o => o.Id == id);
    }

    public bool ActualizarClienteConClienteGana(OrdenFabricacion orden, string nuevoCliente)
    {
        orden.Cliente = nuevoCliente;

        try
        {
            _context.SaveChanges();
            return true;
        }
        catch (DbUpdateConcurrencyException ex)
        {
            var entry = ex.Entries.First();
            var valoresBd = entry.GetDatabaseValues();

            if (valoresBd is null) return false;

            entry.OriginalValues.SetValues(valoresBd);
            _context.SaveChanges();
            return true;
        }
    }

    public bool InsertarDosOrdenesConTransaccion(string numero1, string numero2)
    {
        using var transaction = _context.Database.BeginTransaction();
    }

```

```

        try
        {
            _context.OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = numero1, Cliente =
"Norte", Estado = "Pendiente", FechaCreacion = DateTime.Now });
            _context.SaveChanges();

            _context.OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = numero2, Cliente =
"Sur", Estado = "Pendiente", FechaCreacion = DateTime.Now });
            _context.SaveChanges();

            transaction.Commit();
            return true;
        }
        catch (DbUpdateException ex)
        {
            transaction.Rollback();
            _logger.LogError(ex, "Error al insertar las órdenes");
            return false;
        }
    }

    public void AplicarMigracionHasta(string nombreMigracion)
    {
        var migrator =
            _context.Database.GetService<Microsoft.EntityFrameworkCore.Migrations.IMigrator>();
        migrator.Migrate(nombreMigracion);
    }
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 3: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 5: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 6: using Microsoft.Extensions.Logging; → importa el logging.

Línea 8: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 10: public class OrdenRepositorio : Repositorio, IOrdenRepositorio → declara la implementación específica que hereda de la genérica.

Línea 12: private readonly ILogger _logger; → campo del logger.

Línea 14: public OrdenRepositorio(AceriaDbContext context, ILogger logger) : base(context) → constructor.

Línea 16: _logger = logger; → asigna el logger.

Error común: si el repositorio específico llama a _context.SaveChanges(), rompe el patrón de unidad de trabajo. El repositorio no debe guardar cambios: eso lo hace la unidad de trabajo.

| Paso 6: Refactorizar UnidadDeTrabajo con IDbContextFactory

Modificar src/AceriaData.Infrastructure.Repositories/UnidadDeTrabajo.cs:

```

using AceriaData.Application.Interfaces;
using AceriaData.Infrastructure.Persistence;

namespace AceriaData.Infrastructure.Repositories;

public class UnidadDeTrabajo : IUnidadDeTrabajo
{
    private readonly AceriaDbContext _context;
    private IOrdenRepositorio? _ordenes;
    private IPlanchaRepositorio? _planchas;
    private IAleacionRepositorio? _aleaciones;
    private IDetalleOrdenRepositorio? _detalles;

    public UnidadDeTrabajo(AceriaDbContext context)
    {
        _context = context;
    }

    public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context,
    NullLogger<OrdenRepositorio>.Instance);
    public IPlanchaRepositorio Planchas => _planchas ??= new PlanchaRepositorio(_context);
    public IAleacionRepositorio Aleaciones => _aleaciones ??= new AleacionRepositorio(_context);
    public IDetalleOrdenRepositorio Detalles => _detalles ??= new DetalleOrdenRepositorio(_context);

    public int Guardar()
    {
        return _context.SaveChanges();
    }

    public void Dispose()
    {
        _context.Dispose();
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 2: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 4: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 6: public class UnidadDeTrabajo : IUnidadDeTrabajo → declara la implementación.

Línea 8: private readonly AceriaDbContext _context; → campo del DbContext.

Línea 9: private IOrdenRepositorio? _ordenes; → campo del repositorio de órdenes.

Línea 10: private IPlanchaRepositorio? _planchas; → campo del repositorio de planchas.

Línea 11: private IAleacionRepositorio? _aleaciones; → campo del repositorio de aleaciones.

Línea 12: private IDetalleOrdenRepositorio? _detalles; → campo del repositorio de detalles.

Línea 14: public UnidadDeTrabajo(AceriaDbContext context) → constructor.

Línea 16: _context = context; → asigna el parámetro al campo.

Línea 19: public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context, NullLogger.Instance); → expone el repositorio de órdenes.

Línea 20: public IPlanchaRepositorio Planchas => _planchas ??= new PlanchaRepositorio(_context); → expone el repositorio de planchas.

Línea 21: public IAleacionRepositorio Aleaciones => _aleaciones ??= new AleacionRepositorio(_context); → expone el repositorio de aleaciones.

Línea 22: public IDetalleOrdenRepositorio Detalles => _detalles ??= new DetalleOrdenRepositorio(_context); → expone el repositorio de detalles.

Línea 24: public int Guardar() → declara el método.

Línea 26: `return _context.SaveChanges();` → ejecuta `SaveChanges`.

Línea 29: `public void Dispose()` → declara el método.

Línea 31: `_context.Dispose();` → libera el `DbContext`.

Error común: si la unidad de trabajo crea los repositorios en el constructor en lugar de usar inicialización perezosa, se crean todos los repositorios aunque no se usen.

| Paso 7: Crear un servicio de larga duración con `IDbContextFactory`

Crear el archivo `src/AceriaData.Infrastructure.Services/ProcesadorOrdenes.cs`:

```
using AceriaData.Infrastructure.Persistence;
using AceriaData.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Hosting;

namespace AceriaData.Infrastructure.Services;

public class ProcesadorOrdenes : BackgroundService
{
    private readonly IDbContextFactory<AceriaDbContext> _factory;

    public ProcesadorOrdenes(IDbContextFactory<AceriaDbContext> factory)
    {
        _factory = factory;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            try
            {
                using var context = await _factory.CreateDbContextAsync(stoppingToken);
                using var unidad = new UnidadDeTrabajo(context);

                var pendientes = unidad Ordenes.ObtenerPendientes();
                Console.WriteLine($"Órdenes pendientes: {pendientes.Count}");

                await Task.Delay(TimeSpan.FromSeconds(5), stoppingToken);
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error en el procesador: {ex.Message}");
                await Task.Delay(TimeSpan.FromSeconds(10), stoppingToken);
            }
        }
    }
}
```

Línea 1: `using AceriaData.Infrastructure.Persistence;` → importa el `DbContext`.

Línea 2: `using AceriaData.Infrastructure.Repositories;` → importa la unidad de trabajo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa EF Core.

Línea 4: `using Microsoft.Extensions.Hosting;` → importa el host.

Línea 6: `namespace AceriaData.Infrastructure.Services;` → declara el espacio de nombres.

Línea 8: `public class ProcesadorOrdenes : BackgroundService` → declara el servicio.

Línea 10: `private readonly IDbContextFactory _factory;` → campo de la fábrica.

Línea 12: `public ProcesadorOrdenes(IDbContextFactory factory)` → constructor.

Línea 14: `_factory = factory;` → asigna la fábrica.

Línea 17: `protected override async Task ExecuteAsync(CancellationToken stoppingToken)` → declara el método.

Línea 19: `while (!stoppingToken.IsCancellationRequested)` → bucle hasta la cancelación.

Línea 21: `try` → inicio del bloque.

Línea 23: `using var context = await _factory.CreateDbContextAsync(stoppingToken);` → crea un contexto desde la fábrica.

Línea 24: `using var unidad = new UnidadDeTrabajo(context);` → crea la unidad de trabajo.

Línea 26: `var pendientes = unidad Ordenes.ObtenerPendientes();` → obtiene las órdenes pendientes.

Línea 27: `Console.WriteLine($"Órdenes pendientes: {pendientes.Count}");` → muestra el número.

Línea 29: `await Task.Delay(TimeSpan.FromSeconds(5), stoppingToken);` → espera cinco segundos.

Línea 31: `catch (Exception ex)` → captura la excepción.

Línea 33: `Console.WriteLine($"Error en el procesador: {ex.Message}");` → muestra el mensaje.

Línea 34: `await Task.Delay(TimeSpan.FromSeconds(10), stoppingToken);` → espera diez segundos.

Error común: si se inyecta el `DbContext` directamente en el `BackgroundService`, el contexto vive más que el ámbito para el que fue creado y se producen errores. Se debe usar `IDbContextFactory`.

Paso 8: Registrar los servicios en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddDbContextFactory<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sqlOptions =>
            {
                sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                sqlOptions.CommandTimeout(60);
                sqlOptions.MigrationsAssembly("AceriaData.Infrastructure");
                // Se conserva la tabla predeterminada __EFMigrationsHistory y el historial heredado.
            });

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IPlanchaRepositorio, PlanchaRepositorio>();
services.AddScoped<IAleacionRepositorio, AleacionRepositorio>();
services.AddScoped<IDetalleOrdenRepositorio, DetalleOrdenRepositorio>();
services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
services.AddHostedService<ProcesadorOrdenes>();
```

Línea 1: `services.AddDbContextFactory(options =>` → registra la fábrica de `DbContext`.

Línea 2: `options` → objeto de configuración.

Línea 3: `.UseSqlServer(connectionString, sqlOptions =>` → registra el proveedor.

Línea 4: `{` → inicio del bloque.

Línea 5: `sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);` → habilita los reintentos.

Línea 6: `sqlOptions.CommandTimeout(60);` → establece el tiempo de espera.

Línea 7: `sqlOptions.MigrationsAssembly("AceriaData.Infrastructure");` → especifica el ensamblado de migraciones.

Línea 8: se conserva `__EFMigrationsHistory` → el módulo no cambia arbitrariamente la tabla ni pierde la cadena histórica existente.

Línea 9: `});` → cierra el bloque.

Línea 11: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra el repositorio de órdenes.

Línea 12: `services.AddScoped<IPlanchaRepositorio, PlanchaRepositorio>();` → registra el repositorio de planchas.

Línea 13: `services.AddScoped<IAleacionRepositorio, AleacionRepositorio>();` → registra el repositorio de aleaciones.

Línea 14: `services.AddScoped<IDetalleOrdenRepositorio, DetalleOrdenRepositorio>();` → registra el repositorio de detalles.

Línea 15: `services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();` → registra la unidad de trabajo.

Línea 16: `services.AddHostedService();` → registra el servicio de larga duración.

Error común: si se registra el `DbContext` con `AddDbContext` en lugar de `AddDbContextFactory`, el `BackgroundService` no puede resolver el contexto porque su ciclo de vida es `Scoped` y el servicio es `Singleton`. Se debe usar `AddDbContextFactory`.

▮ Paso 9: Crear el caso de uso de patrón Repositorio y Unidad de Trabajo

Corrección del benchmark original

Los métodos históricos `CompararRendimientoConYSinRepositorio` y `CompararMemoriaConYSinRepositorio` se conservan para trazabilidad con la fuente, pero **no constituyen un benchmark válido de acceso directo frente a Repository**: los caminos del ejemplo terminan usando la misma abstracción.

`GC.GetTotalMemory` tampoco mide de forma precisa el overhead atribuible al patrón. Cualquier cifra observada se trata únicamente como una medición local del proceso.

Crear el archivo `src/AceriaData.Application/UseCases/RepositorioUnidadTrabajoUseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.UseCases;

public class RepositorioUnidadTrabajoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public RepositorioUnidadTrabajoUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PATRÓN REPOSITORIO Y UNIDAD DE TRABAJO ===");

        DemostrarRepositorioGenerico();
        DemostrarRepositorioEspecifico();
        DemostrarUnidadDeTrabajo();
        CompararRendimientoConYSinRepositorio();
        CompararMemoriaConYSinRepositorio();
        MostrarAntiPatrones();
    }

    private void DemostrarRepositorioGenerico()
    {
        Console.WriteLine("\n--- Repositorio genérico ---");

        var todas = _unidad.Ordenes.ObtenerTodas();
        var porId = _unidad.Ordenes.ObtenerPorId(1);

        Console.WriteLine($"Todas las órdenes: {todas.Count}");
        Console.WriteLine($"Orden con Id 1: {porId?.NumeroOrden}");
    }
}
```

```

private void DemostrarRepositorioEspecifico()
{
    Console.WriteLine("\n--- Repositorio específico ---");

    var pendientes = _unidad.Ordenes.ObtenerPendientes();
    var resúmenes = _unidad.Ordenes.ObtenerResúmenes();
    var total = _unidad.Ordenes.ContarOrdenes();

    Console.WriteLine($"Órdenes pendientes: {pendientes.Count}");
    Console.WriteLine($"Resúmenes: {resúmenes.Count}");
    Console.WriteLine($"Total de órdenes: {total}");
}

private void DemostrarUnidadDeTrabajo()
{
    Console.WriteLine("\n--- Unidad de trabajo ---");

    var orden = new OrdenFabricacion
    {
        NumeroOrden = "OF-UOW-001",
        Cliente = "Constructora Unidad",
        Estado = "Pendiente",
        FechaCreacion = DateTime.Now
    };

    _unidad.Ordenes.Agregar(orden);

    var plancha = new PlanchaAcero
    {
        Orden = orden,
        Espesor = 10.5,
        Ancho = 1500,
        Largo = 3000,
        Peso = 370.5m,
        Activa = true
    };

    _unidad.Planchas.Agregar(plancha);

    var filas = _unidad.Guardar();
    Console.WriteLine($"Filas afectadas: {filas}");
    Console.WriteLine($"Orden creada: {orden.NumeroOrden} con Id {orden.Id}");
}

private void CompararRendimientoConYSinRepositorio()
{
    Console.WriteLine("\n--- Comparación de rendimiento con y sin repositorio ---");

    var cronometroSin = Stopwatch.StartNew();
    for (int i = 0; i < 100; i++)
    {
        var ordenes = _unidad.Ordenes.ObtenerPendientes();
    }
    cronometroSin.Stop();

    var cronometroCon = Stopwatch.StartNew();
    for (int i = 0; i < 100; i++)
    {
        var ordenes = _unidad.Ordenes.ObtenerPendientes();
    }
    cronometroCon.Stop();

    Console.WriteLine($"100 consultas con repositorio: {cronometroSin.ElapsedMilliseconds} ms");
    Console.WriteLine($"100 consultas con repositorio (mismo método): {cronometroCon.ElapsedMilliseconds} ms");
}

private void CompararMemoriaConYSinRepositorio()
{
    Console.WriteLine("\n--- Comparación de memoria con y sin repositorio ---");

    GC.Collect();
}

```

```

        GC.WaitForPendingFinalizers();
        GC.Collect();

        var memoriaAntes = GC.GetTotalMemory(true);

        for (int i = 0; i < 1000; i++)
        {
            var repositorio = _unidad.Ordenes;
            var total = repositorio.ContarOrdenes();
        }

        GC.Collect();
        GC.WaitForPendingFinalizers();
        GC.Collect();

        var memoriaDespues = GC.GetTotalMemory(true);
        var diferencia = (memoriaDespues - memoriaAntes) / 1024;

        Console.WriteLine($"Memoria antes: {memoriaAntes / 1024} KB");
        Console.WriteLine($"Memoria después: {memoriaDespues / 1024} KB");
        Console.WriteLine($"Diferencia: {diferencia} KB");
    }

    private void MostrarAntiPatrones()
    {
        Console.WriteLine("\n--- Anti-patrones del patrón Repositorio ---");
        Console.WriteLine("1. Exponer IQueryable en lugar de listas.");
        Console.WriteLine("2. Exponer operaciones de EF Core (Include, AsNoTracking) en la interfaz.");
        Console.WriteLine("3. Repositorio genérico excesivo con métodos para todas las operaciones.");
        Console.WriteLine("4. Repositorio que llama a SaveChanges en lugar de la unidad de trabajo.");
        Console.WriteLine("5. Repositorio que devuelve entidades desconectadas sin control.");
        Console.WriteLine();
        Console.WriteLine("Buenas prácticas:");
        Console.WriteLine("1. Devolver listas o entidades concretas.");
        Console.WriteLine("2. Encapsular las consultas específicas del dominio.");
        Console.WriteLine("3. Añadir solo los métodos que la capa de negocio necesita.");
        Console.WriteLine("4. El repositorio no guarda: la unidad de trabajo coordina.");
        Console.WriteLine("5. Documentar las decisiones de acceso a datos.");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces.

Línea 3: using AceriaData.Domain.Entities; → importa las entidades.

Línea 5: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 7: public class RepositorioUnidadTrabajoUseCase → declara el caso de uso.

Línea 9: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 11: public RepositorioUnidadTrabajoUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 13: _unidad = unidad; → asigna el parámetro al campo.

Línea 16: public void Ejecutar() → declara el método principal.

Línea 18: Console.WriteLine("=== PATRÓN REPOSITORIO Y UNIDAD DE TRABAJO ==="); → muestra la cabecera.

Línea 20: DemostrarRepositorioGenerico(); → llama al método de repositorio genérico.

Línea 21: DemostrarRepositorioEspecifico(); → llama al método de repositorio específico.

Línea 22: DemostrarUnidadDeTrabajo(); → llama al método de unidad de trabajo.

Línea 23: CompararRendimientoConYSinRepositorio(); → llama al método de comparación de rendimiento.

Línea 24: CompararMemoriaConYSinRepositorio(); → llama al método de comparación de memoria.

Línea 25: `MostrarAntiPatrones();` → llama al método de anti-patrones.

Línea 28: `private void DemostrarRepositorioGenerico()` → declara el método.

Línea 30: `Console.WriteLine("\n--- Repositorio genérico ---");` → muestra la cabecera.

Línea 32: `var todas = _unidad Ordenes.ObtenerTodas();` → llama al método genérico.

Línea 33: `var porId = _unidad Ordenes.ObtenerPorId(1);` → llama al método genérico.

Línea 35: `Console.WriteLine($"Todas las órdenes: {todas.Count}");` → muestra el número.

Línea 36: `Console.WriteLine($"Orden con Id 1: {porId?.NumeroOrden}");` → muestra el número de orden.

Línea 39: `private void DemostrarRepositorioEspecifico()` → declara el método.

Línea 41: `Console.WriteLine("\n--- Repositorio específico ---");` → muestra la cabecera.

Línea 43: `var pendientes = _unidad Ordenes.ObtenerPendientes();` → llama al método específico.

Línea 44: `var resúmenes = _unidad Ordenes.ObtenerResúmenes();` → llama al método específico.

Línea 45: `var total = _unidad Ordenes.ContarOrdenes();` → llama al método específico.

Línea 47: `Console.WriteLine($"Órdenes pendientes: {pendientes.Count}");` → muestra el número.

Línea 48: `Console.WriteLine($"Resúmenes: {resúmenes.Count}");` → muestra el número.

Línea 49: `Console.WriteLine($"Total de órdenes: {total}");` → muestra el total.

Línea 52: `private void DemostrarUnidadDeTrabajo()` → declara el método.

Línea 54: `Console.WriteLine("\n--- Unidad de trabajo ---");` → muestra la cabecera.

Línea 56: `var orden = new OrdenFabricacion` → crea la orden.

Línea 64: `_unidad Ordenes.Agregar(orden);` → agrega la orden.

Línea 66: `var plancha = new PlanchaAcero` → crea la plancha.

Línea 76: `_unidad Planchas.Agregar(plancha);` → agrega la plancha.

Línea 78: `var filas = _unidad.Guardar();` → guarda los cambios.

Línea 79: `Console.WriteLine($"Filas afectadas: {filas}");` → muestra el número de filas.

Línea 80: `Console.WriteLine($"Orden creada: {orden.NumeroOrden} con Id {orden.Id}");` → muestra la orden creada.

Línea 83: `private void CompararRendimientoConYSinRepositorio()` → declara el método.

Línea 85: `Console.WriteLine("\n--- Comparación de rendimiento con y sin repositorio ---");` → muestra la cabecera.

Línea 87: `var cronometroSin = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 88: `for (int i = 0; i < 100; i++)` → repite cien veces.

Línea 90: `var ordenes = _unidad Ordenes.ObtenerPendientes();` → llama al método del repositorio.

Línea 92: `cronometroSin.Stop();` → detiene el cronómetro.

Línea 94: `var cronometroCon = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 95: `for (int i = 0; i < 100; i++)` → repite cien veces.

Línea 97: `var ordenes = _unidad Ordenes.ObtenerPendientes();` → llama al método del repositorio.

Línea 99: `cronometroCon.Stop();` → detiene el cronómetro.

Línea 101: `Console.WriteLine($"100 consultas con repositorio: {cronometroSin.ElapsedMilliseconds} ms");` → muestra el tiempo.

Línea 102: `Console.WriteLine($"100 consultas con repositorio (mismo método): {cronometroCon.ElapsedMilliseconds} ms");` → muestra el tiempo.

Línea 105: `private void CompararMemoriaConYSinRepositorio()` → declara el método.

Línea 107: `Console.WriteLine("\n--- Comparación de memoria con y sin repositorio ---");` → muestra la cabecera.

Línea 109: `GC.Collect();` → recolecta la basura.

Línea 110: `GC.WaitForPendingFinalizers();` → espera a los finalizadores.

Línea 111: `GC.Collect();` → recolecta de nuevo.

Línea 113: `var memoriaAntes = GC.GetTotalMemory(true);` → obtiene la memoria antes.

Línea 115: `for (int i = 0; i < 1000; i++)` → repite mil veces.

Línea 117: `var repositorio = _unidad Ordenes;` → obtiene el repositorio.

Línea 118: `var total = repositorio.ContarOrdenes();` → cuenta las órdenes.

Línea 121: `GC.Collect();` → recolecta la basura.

Línea 122: `GC.WaitForPendingFinalizers();` → espera a los finalizadores.

Línea 123: `GC.Collect();` → recolecta de nuevo.

Línea 125: `var memoriaDespues = GC.GetTotalMemory(true);` → obtiene la memoria después.

Línea 126: `var diferencia = (memoriaDespues - memoriaAntes) / 1024;` → calcula la diferencia.

Línea 128: `Console.WriteLine($"Memoria antes: {memoriaAntes / 1024} KB");` → muestra la memoria antes.

Línea 129: `Console.WriteLine($"Memoria después: {memoriaDespues / 1024} KB");` → muestra la memoria después.

Línea 130: `Console.WriteLine($"Diferencia: {diferencia} KB");` → muestra la diferencia.

Línea 133: `private void MostrarAntiPatrones()` → declara el método.

Línea 135: `Console.WriteLine("\n--- Anti-patrones del patrón Repositorio ---");` → muestra la cabecera.

Línea 136: `Console.WriteLine("1. Exponer IQueryable en lugar de listas.");` → describe el primer anti-patrón.

Línea 137: `Console.WriteLine("2. Exponer operaciones de EF Core (Include, AsNoTracking) en la interfaz.");` → describe el segundo.

Línea 138: `Console.WriteLine("3. Repositorio genérico excesivo con métodos para todas las operaciones.");` → describe el tercero.

Línea 139: `Console.WriteLine("4. Repositorio que llama a SaveChanges en lugar de la unidad de trabajo.");` → describe el cuarto.

Línea 140: `Console.WriteLine("5. Repositorio que devuelve entidades desconectadas sin control.");` → describe el quinto.

Línea 142: `Console.WriteLine("Buenas prácticas:");` → muestra la cabecera de buenas prácticas.

Línea 143: `Console.WriteLine("1. Devolver listas o entidades concretas.");` → describe la primera.

Línea 144: `Console.WriteLine("2. Encapsular las consultas específicas del dominio.");` → describe la segunda.

Línea 145: `Console.WriteLine("3. Añadir solo los métodos que la capa de negocio necesita.");` → describe la tercera.

Línea 146: `Console.WriteLine("4. El repositorio no guarda: la unidad de trabajo coordina.");` → describe la cuarta.

Línea 147: `Console.WriteLine("5. Documentar las decisiones de acceso a datos.");` → describe la quinta.

Error común: si se llama a Guardar varias veces, se ejecutan varias transacciones. Se debe llamar una sola vez al final.

Paso 10: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<RepositorioUnidadTrabajoUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

Paso 11: Insertar datos de prueba iniciales

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 3, 10) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var ordenes = new List` → crea la lista.

Línea 9: `new OrdenFabricacion { ... }` → primera orden.

Línea 10: `new OrdenFabricacion { ... }` → segunda orden.

Línea 11: `new OrdenFabricacion { ... }` → tercera orden.

Línea 12: `};` → cierra la lista.

Línea 14: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 15: `context.SaveChanges();` → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 12: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados del repositorio genérico, el repositorio específico, la unidad de trabajo, la comparación de rendimiento, la comparación de memoria y los anti-patrones.

Paso 13: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== PATRÓN REPOSITORIO Y UNIDAD DE TRABAJO ===

--- Repositorio genérico ---
Todas las órdenes: 3
Orden con Id 1: OF-2024-0001

--- Repositorio específico ---
Órdenes pendientes: 2
Resúmenes: 3
Total de órdenes: 3

--- Unidad de trabajo ---
Filas afectadas: 2
Orden creada: OF-UOW-001 con Id 4

--- Observación local del mismo camino de Repository ---
Primera ejecución: <medición local>
Segunda ejecución del mismo método: <medición local>
No constituye una comparación "directo vs Repository".

--- Observación local de memoria del proceso ---
Memoria antes: <medición local>
Memoria después: <medición local>
Diferencia: <medición local>
GC.GetTotalMemory no aísla el overhead de Repository.

--- Anti-patrones del patrón Repositorio ---
1. Exponer IQueryable en lugar de listas.
2. Exponer operaciones de EF Core (Include, AsNoTracking) en la interfaz.
3. Repositorio genérico excesivo con métodos para todas las operaciones.
4. Repositorio que llama a SaveChanges en lugar de la unidad de trabajo.
5. Repositorio que devuelve entidades desconectadas sin control.

Buenas prácticas:
1. Devolver listas o entidades concretas.
2. Encapsular las consultas específicas del dominio.
3. Añadir solo los métodos que la capa de negocio necesita.
4. El repositorio no guarda: la unidad de trabajo coordina.
5. Documentar las decisiones de acceso a datos.
La primera sección muestra el repositorio genérico. La segunda sección muestra el repositorio específico. La tercera sección muestra la unidad de trabajo. La cuarta sección muestra la comparación de rendimiento. La quinta sección muestra la comparación de memoria. La sexta sección muestra los anti-patrones.

Observaciones: el repositorio genérico proporciona métodos comunes y el repositorio específico añade operaciones del dominio. La unidad de trabajo coordina los repositorios. Las cifras de tiempo y memoria de una ejecución local no demuestran el overhead de Repository; el ejemplo original no compara dos caminos arquitectónicos realmente distintos y `GC.GetTotalMemory` no aísla el coste del patrón.
```

Paso 14: Diagnosticar un error común

Modificar el repositorio para exponer IQueryable:

```
public interface IOordenRepositorio : IRepository<OrdenFabricacion>
{
    IQueryable<OrdenFabricacion> ObtenerQueryable();
}
```

Resultado esperado: la interfaz expone IQueryable, lo que rompe la abstracción. La capa de negocio puede añadir filtros y ordenaciones que el repositorio no controla.

Solución: devolver listas o entidades concretas en lugar de IQueryable.

```
public interface IOordenRepositorio : IRepository<OrdenFabricacion>
{
    List<OrdenFabricacion> ObtenerPendientes();
    List<OrdenFabricacion> ObtenerPorCliente(string cliente);
}
```

Resultado esperado con la solución: la interfaz devuelve listas y mantiene la abstracción.

| Paso 15: Crear un test con Moq

Crear el archivo tests/AceriaData.Tests/Repositories/OrdenRepositorioMockTests.cs:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using Moq;

namespace AceriaData.Tests.Repositories;

public class OrdenRepositorioMockTests
{
    [Fact]
    public void ObtenerPendientes_ConVariasOrdenes_DevuelveSoloLasPendientes()
    {
        var mockRepositorio = new Mock<IOordenRepositorio>();
        mockRepositorio.Setup(r => r.ObtenerTodas()).Returns(new List<OrdenFabricacion>
        {
            new OrdenFabricacion { NumeroOrden = "OF-001", Estado = "Pendiente" },
            new OrdenFabricacion { NumeroOrden = "OF-002", Estado = "EnProceso" },
            new OrdenFabricacion { NumeroOrden = "OF-003", Estado = "Pendiente" }
        });

        var ordenes = mockRepositorio.Object.ObtenerTodas().Where(o => o.Estado ==
"Pendiente").ToList();

        Assert.Equal(2, ordenes.Count);
    }

    [Fact]
    public void Agregar_ConOrdenValida_LlamaAlMetodoDelRepositorio()
    {
        var mockRepositorio = new Mock<IOordenRepositorio>();
        var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Estado = "Pendiente" };

        mockRepositorio.Object.Agregar(orden);

        mockRepositorio.Verify(r => r.Agregar(orden), Times.Once);
    }
}
```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 3: using Moq; → importa Moq.

Línea 5: namespace AceriaData.Tests.Repositories; → declara el espacio de nombres.

Línea 7: public class OrdenRepositorioMockTests → declara la clase de tests.

Línea 9: [Fact] → atributo de test.

Línea 10: `public void ObtenerPendientes_ConVariasOrdenes_DevuelveSoloLasPendientes()` → declara el test.

Línea 12: `var mockRepositorio = new Mock();` → crea el mock.

Línea 13: `mockRepositorio.Setup(r => r.ObtenerTodas()).Returns(new List` → configura el método.

Línea 19: `var ordenes = mockRepositorio.Object.ObtenerTodas().Where(o => o.Estado == "Pendiente").ToList();` → llama al método.

Línea 21: `Assert.Equal(2, ordenes.Count);` → comprueba el número.

Línea 24: [Fact] → atributo de test.

Línea 25: `public void Agregar_ConOrdenValida_LlamaAlMetodoDelRepositorio()` → declara el test.

Línea 27: `var mockRepositorio = new Mock();` → crea el mock.

Línea 28: `var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Estado = "Pendiente" };` → crea la orden.

Línea 30: `mockRepositorio.Object.Agregar(orden);` → llama al método.

Línea 32: `mockRepositorio.Verify(r => r.Agregar(orden), Times.Once);` → comprueba que se llamó una vez.

Error común: si no se instala el paquete Moq, el código no compila. Se debe añadir con `dotnet add package Moq`.

Paso 16: Ejecutar los tests

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj
```

`dotnet test` → ejecuta los tests del proyecto.

Resultado esperado: todos los tests pasan.

La salida del comando muestra información como la siguiente:

```
Passed! - Failed: 0, Passed: 2, Skipped: 0, Total: 2, Duration: 1 s
Observaciones: los tests con Moq no necesitan la base de datos. Verifican el comportamiento del
repositorio de forma aislada.
```

Errores comunes del ejercicio

Error Causa Solución Repositorio llama a SaveChanges Rompe la unidad de trabajo El repositorio no guarda: solo la unidad de trabajo Repositorio expone IQueryable Rompe la abstracción Devolver listas o entidades concretas Repositorio expone Include Acopla la capa de negocio a EF Core Usar métodos específicos del dominio Registro como Singleton Problemas de concurrencia Registrar con Scoped DbContext compartido entre ámbitos Datos obsoletos Usar CreateScope para cada operación Olvidar Guardar Los cambios no se persisten Llamar a unidad.Guardar() Repositorio genérico demasiado amplio Se convierte en una envoltura de DbSet Añadir solo los métodos necesarios BackgroundService con DbContext inyectado El contexto vive más que el ámbito Usar IDbContextFactory Mock sin configurar Los métodos devuelven null Configurar con Setup Moq no instalado Falta el paquete Añadir con `dotnet add package Moq`

Reto resuelto: Añadir un repositorio específico para DetalleOrden con mocking

Reto: Crear la interfaz IDetalleOrdenRepositorio y su implementación DetalleOrdenRepositorio. Añadirla a la unidad de trabajo. Crear un test con Moq que verifique el método ObtenerPorOrden. Verificar que funciona con un caso de uso.

Solución paso a paso:

Paso 1: Crear la interfaz IDetalleOrdenRepositorio:

```
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IDetalleOrdenRepositorio : IRepository<DetalleOrden>
{
    DetalleOrden? ObtenerPorOrden(int ordenId);
    List<DetalleOrden> ObtenerConTemperaturaMayorA(double temperatura);
}
```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 3: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 5: public interface IDetalleOrdenRepositorio : IRepository → declara la interfaz específica.

Línea 7: DetalleOrden? ObtenerPorOrden(int ordenId); → declara el método.

Línea 8: List ObtenerConTemperaturaMayorA(double temperatura); → declara el método.

Paso 2: Crear la implementación DetalleOrdenRepositorio:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;

namespace AceriaData.Infrastructure.Repositories;

public class DetalleOrdenRepositorio : Repository<DetalleOrden>, IDetalleOrdenRepositorio
{
    public DetalleOrdenRepositorio(AceriaDbContext context) : base(context) { }

    public DetalleOrden? ObtenerPorOrden(int ordenId)
    {
        return _context.DetallesOrden
            .FirstOrDefault(d => d.OrdenId == ordenId);
    }

    public List<DetalleOrden> ObtenerConTemperaturaMayorA(double temperatura)
    {
        return _context.DetallesOrden
            .Where(d => d.TemperaturaColada > temperatura)
            .ToList();
    }
}
```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 3: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 5: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 7: public class DetalleOrdenRepositorio : Repository, IDetalleOrdenRepositorio → declara la implementación.

Línea 9: `public DetalleOrdenRepositorio(AceriaDbContext context) : base(context) { }` → constructor.

Línea 11: `public DetalleOrden? ObtenerPorOrden(int ordenId)` → declara el método.

Línea 13: `return _context.DetallesOrden` → inicia la consulta.

Línea 14: `.FirstOrDefault(d => d.OrdenId == ordenId);` → filtra por orden.

Línea 17: `public List ObtenerConTemperaturaMayorA(double temperatura)` → declara el método.

Línea 19: `return _context.DetallesOrden` → inicia la consulta.

Línea 20: `.Where(d => d.TemperaturaColada > temperatura)` → filtra por temperatura.

Línea 21: `.ToList();` → materializa la consulta.

Paso 3: Añadir el repositorio a la unidad de trabajo:

```
public interface IUnidadDeTrabajo : IDisposable
{
    IOrdenRepositorio Ordenes { get; }
    IPlanchaRepositorio Planchas { get; }
    IAleacionRepositorio Aleaciones { get; }
    IDetalleOrdenRepositorio Detalles { get; }
    int Guardar();
}
```

Línea 1: `public interface IUnidadDeTrabajo : IDisposable` → declara la interfaz.

Línea 3: `IOrdenRepositorio Ordenes { get; }` → expone el repositorio de órdenes.

Línea 4: `IPlanchaRepositorio Planchas { get; }` → expone el repositorio de planchas.

Línea 5: `IAleacionRepositorio Aleaciones { get; }` → expone el repositorio de aleaciones.

Línea 6: `IDetalleOrdenRepositorio Detalles { get; }` → expone el repositorio de detalles.

Línea 7: `int Guardar();` → declara el método que guarda los cambios.

Paso 4: Implementar el repositorio en la unidad de trabajo:

```
public class UnidadDeTrabajo : IUnidadDeTrabajo
{
    private readonly AceriaDbContext _context;
    private IOrdenRepositorio? _ordenes;
    private IPlanchaRepositorio? _planchas;
    private IAleacionRepositorio? _aleaciones;
    private IDetalleOrdenRepositorio? _detalles;

    public UnidadDeTrabajo(AceriaDbContext context)
    {
        _context = context;
    }

    public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context,
        NullLogger<OrdenRepositorio>.Instance);
    public IPlanchaRepositorio Planchas => _planchas ??= new PlanchaRepositorio(_context);
    public IAleacionRepositorio Aleaciones => _aleaciones ??= new AleacionRepositorio(_context);
    public IDetalleOrdenRepositorio Detalles => _detalles ??= new DetalleOrdenRepositorio(_context);

    public int Guardar() => _context.SaveChanges();

    public void Dispose() => _context.Dispose();
}
```

Línea 1: `public class UnidadDeTrabajo : IUnidadDeTrabajo` → declara la implementación.

Línea 3: `private readonly AceriaDbContext _context;` → campo del DbContext.

Línea 4: `private IOrdenRepositorio? _ordenes;` → campo del repositorio de órdenes.

Línea 5: `private IPlanchaRepositorio? _planchas;` → campo del repositorio de planchas.

Línea 6: `private IAleacionRepositorio? _aleaciones;` → campo del repositorio de aleaciones.

Línea 7: `private IDetalleOrdenRepositorio? _detalles;` → campo del repositorio de detalles.

Línea 9: `public UnidadDeTrabajo(AceriaDbContext context)` → constructor.

Línea 11: `_context = context;` → asigna el parámetro al campo.

Línea 14: `public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context, NullLogger.Instance);` → expone el repositorio de órdenes.

Línea 15: `public IPlanchaRepositorio Planchas => _planchas ??= new PlanchaRepositorio(_context);` → expone el repositorio de planchas.

Línea 16: `public IAleacionRepositorio Aleaciones => _aleaciones ??= new AleacionRepositorio(_context);` → expone el repositorio de aleaciones.

Línea 17: `public IDetalleOrdenRepositorio Detalles => _detalles ??= new DetalleOrdenRepositorio(_context);` → expone el repositorio de detalles.

Línea 19: `public int Guardar() => _context.SaveChanges();` → guarda los cambios.

Línea 21: `public void Dispose() => _context.Dispose();` → libera el DbContext.

Paso 5: Registrar el repositorio en el contenedor:

```
services.AddScoped<IDetalleOrdenRepositorio, DetalleOrdenRepositorio>();
```

Línea 1: `services.AddScoped<IDetalleOrdenRepositorio, DetalleOrdenRepositorio>();` → registra el repositorio.

Paso 6: Crear el test con Moq:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using Moq;

namespace AceriaData.Tests.Repositories;

public class DetalleOrdenRepositorioMockTests
{
    [Fact]
    public void ObtenerPorOrden_ConDetalleExistente_DevuelveElDetalle()
    {
        var mockRepositorio = new Mock<IDetalleOrdenRepositorio>();
        mockRepositorio.Setup(r => r.ObtenerPorOrden(1)).Returns(new DetalleOrden
        {
            Id = 1,
            OrdenId = 1,
            ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
            TemperaturaColada = 1550.5
        });

        var detalle = mockRepositorio.Object.ObtenerPorOrden(1);

        Assert.NotNull(detalle);
        Assert.Equal(1550.5, detalle.TemperaturaColada);
    }

    [Fact]
    public void ObtenerPorOrden_ConOrdenInexistente_DevuelveNull()
    {
        var mockRepositorio = new Mock<IDetalleOrdenRepositorio>();
        mockRepositorio.Setup(r => r.ObtenerPorOrden(99999)).Returns((DetalleOrden?)null);

        var detalle = mockRepositorio.Object.ObtenerPorOrden(99999);

        Assert.Null(detalle);
    }
}
```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 3: using Moq; → importa Moq.

Línea 5: namespace AceriaData.Tests.Repositories; → declara el espacio de nombres.

Línea 7: public class DetalleOrdenRepositorioMockTests → declara la clase de tests.

Línea 9: [Fact] → atributo de test.

Línea 10: public void ObtenerPorOrden_ConDetalleExistente_DevuelveElDetalle() → declara el test.

Línea 12: var mockRepositorio = new Mock(); → crea el mock.

Línea 13: mockRepositorio.Setup(r => r.ObtenerPorOrden(1)).Returns(new DetalleOrden → configura el método.

Línea 21: var detalle = mockRepositorio.Object.ObtenerPorOrden(1); → llama al método.

Línea 23: Assert.NotNull(detalle); → comprueba que existe.

Línea 24: Assert.Equal(1550.5, detalle.TemperaturaColada); → comprueba la temperatura.

Línea 27: [Fact] → atributo de test.

Línea 28: public void ObtenerPorOrden_ConOrdenInexistente_DevuelveNull() → declara el test.

Línea 30: var mockRepositorio = new Mock(); → crea el mock.

Línea 31: `mockRepositorio.Setup(r => r.ObtenerPorOrden(99999)).Returns((DetalleOrden?)null);` → configura el método.

Línea 33: `var detalle = mockRepositorio.Object.ObtenerPorOrden(99999);` → llama al método.

Línea 35: `Assert.Null(detalle);` → comprueba que es nulo.

Paso 7: Ejecutar los tests:

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj
```

Resultado esperado: todos los tests pasan.

Analogía final

El patrón Repositorio y la unidad de trabajo en una acería son como la organización de las ventanillas y el parte de producción. Cada repositorio es una ventanilla especializada: una para órdenes, otra para planchas, otra para aleaciones. La unidad de trabajo es el parte de producción que coordina todas las ventanillas del turno.

El operario no va al almacén directamente: se acerca a la ventanilla y pide lo que necesita. La ventanilla sabe dónde está cada cosa. El parte de producción agrupa todas las operaciones del turno y las confirma al final. Si algo falla, el parte se cancela y ninguna operación se aplica. El repositorio no guarda cambios: eso lo hace el parte.

El `IDbContextFactory` es como un almacén que abre y cierra la puerta cada vez que un operario necesita entrar. No mantiene la puerta abierta durante todo el turno. Cada operario abre la puerta, hace su trabajo y la cierra.

La comparación no debe presentar un overhead fijo como conclusión universal. `DbContext` ya implementa conceptos propios de Unit of Work y Repository; `AceriaData` mantiene una abstracción adicional como decisión arquitectónica deliberada para aislar `Application` y facilitar determinados tests. Si se mide rendimiento o memoria, el resultado se registra como observación local y con un benchmark que compare caminos realmente distintos.

Estas decisiones no son anti-patrones universales por su sola presencia. En `AceriaData` son señales a revisar cuando atraviesan indebidamente la frontera de `Application`, filtran detalles de EF Core o eliminan valor de la abstracción. Por ejemplo, exponer `IQueryable` puede acoplar la capa consumidora al proveedor y permitir composición fuera del repositorio; un repositorio genérico puede ser válido si aporta una abstracción útil.

En `AceriaData` la abstracción Repository/Unit of Work encapsula el acceso a datos como decisión arquitectónica del curso. `DbContext` ya implementa responsabilidades relacionadas con Unit of Work y Repository, por lo que una capa adicional debe justificarse por sus fronteras, casos de uso y estrategia de testing.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado la interfaz genérica `IRepositorio`.

Creado la implementación genérica `Repositorio`.

Refactorizado `IOrdenRepositorio` para heredar de `IRepositorio`.

Refactorizado `OrdenRepositorio` para heredar de `Repositorio` con logger.

Refactorizado `UnidadDeTrabajo` con inicialización perezosa.

Creado el servicio `ProcesadorOrdenes` con `IDbContextFactory`.

Registrado todos los servicios en el contenedor.

Creado el caso de uso RepositorioUnidadTrabajoUseCase.

Ejecutado las demostraciones de repositorio genérico, específico y unidad de trabajo.

Ejecutada una observación local de rendimiento/memoria, sin presentarla como benchmark de acceso directo frente a Repository ni como overhead universal.

Documentado los anti-patrones y las buenas prácticas.

Diagnosticado el error de exponer IQueryable.

Creado el repositorio IDetalleOrdenRepositorio con tests de Moq.

| Conexión con el siguiente punto

En este punto se ha consolidado el patrón Repositorio y la unidad de trabajo en el proyecto AceriaData, aplicando buenas prácticas empresariales. Se ha comprobado que el repositorio encapsula el acceso a datos y que la unidad de trabajo coordina varios repositorios en una sola transacción. Se ha integrado IDbContextFactory para servicios de larga duración. Se ha conservado una observación local de rendimiento y memoria como instrumentación del laboratorio; no demuestra un overhead universal de Repository ni sustituye a un benchmark controlado entre caminos realmente distintos. Se han documentado los anti-patrones y las buenas prácticas. Se ha creado un test con Moq para verificar el comportamiento del repositorio de forma aislada.

En el siguiente punto se estudiará el logging y el diagnóstico en EF Core, con sus implicaciones en la monitorización de la capa de persistencia en aplicaciones empresariales.
